



Pune Vidyarthi Griha's College of Science and Commerce

(Department of Computer Science)

A Practical Report On Implementation and Working of PYTEST TESTING TOOL

Submitted by -

Avishkar Anil Ravlekar

TYBSc Computer Science

Academic Year 2025-2026

CERTIFICATE

This is to certify that **Mr. Avishkar Anil Ravlekar**, a student of **Third Year B.Sc. (Computer Science) of Pune Vidyarthi Griha's College of Science and Commerce, Pune**, has successfully completed the practical assignment entitled:

“Implementation and Working of PyTest Testing Tool”

for the academic year 2025–2026, as per the syllabus requirements of Software Testing Tools.

The student has independently installed the **PyTest** tool, understood its configuration process, executed multiple test cases, and documented the complete procedure with appropriate installation and execution screenshots.

The project work submitted is original and has been carried out sincerely under proper academic supervision.

Date: 02 / 03 / 2026

Prof. Ashwini Talekar Mam
(Subject Teacher /Project Guide)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Prof. Ashwini Talekar**, my subject teacher, for her valuable guidance and support throughout the completion of this practical assignment on “Study and Installation of PyTest Testing Tool.”

I am also thankful to the Department of Computer Science, Pune Vidyarthi Griha’s College of Science and Commerce, Pune, for providing the necessary resources and environment to carry out this work successfully.

This practical assignment has helped me understand the importance of software testing tools and provided practical exposure to installation and execution of test cases using PyTest.

Lastly, I would like to thank my parents and friends for their encouragement and support.

Avishkar Anil Ravlekar
TYBSc Computer Science

INDEX

Sr. No.	Particulars	Page No.
1	Certificate	2
2	Acknowledgement	3
3	Introduaction to PyTest	5
4	Objectives	6
5	Features	7
6	System Requirements	8
7	Download Source	9
8	Installation Process	10
9	Configuration Setup	11
10	Working of the Tool	12
11	Advantages	15
12	Limitations	15
13	Real-World Applications	16
14	Challenges Faced	16
15	Conclusion	17
16	References	18

1.Introduction to PyTest

Software testing plays an essential role in the software development life cycle. It ensures that the developed application performs according to the expected requirements. Without proper testing, software may contain errors, bugs, or unexpected behaviors.

PyTest is a powerful and widely used testing framework in Python that helps developers write simple and effective test cases. It is mainly used for unit testing but can also be extended for functional and integration testing.

PyTest allows writing test functions using simple Python assert statements. It automatically discovers test files and executes them, providing clear output showing passed and failed test cases.

Due to its simplicity and flexibility, PyTest has become one of the most preferred testing frameworks for Python applications.

2.OBJECTIVES

The main objectives of this practical are:

To understand the concept of unit testing.

To study the features of the PyTest framework.

To install and configure PyTest in the system.

To write basic test cases using Python.

To execute test cases using PyTest.

To analyze test results including passed and failed cases.

3.Features

PyTest provides several useful features that make testing easier:

- 1. Simple Syntax** – Test cases can be written using normal Python functions.
- 2. Automatic Test Discovery** – It automatically detects files and functions starting with "test_".
- 3. Detailed Error Reporting** – Provides clear and readable failure messages.
- 4. Support for Fixtures** – Helps in setting up test environments.
- 5. Parameterized Testing** – Allows running the same test with multiple inputs.
- 6. Plugin Support** – Supports various plugins to extend functionality.
- 7. Compatible with CI/CD** – Can be integrated into continuous integration systems.

4. System Requirement

The following system requirements are necessary for installing and running PyTest successfully:

To perform this practical, the following system requirements were needed:

Operating System: Windows 10

Python Version: 3.13.0

Code Editor: Visual Studio Code

Package Manager: pip

Basic knowledge of Python programming

5.Download Resource (Official Website)

PyTest is an open-source testing framework and can be downloaded from its official documentation website.

Official Website: <https://docs.pytest.org>

PyTest can be installed using the Python package manager (pip) with the following command:

```
pip install pytest
```

The above command downloads and installs the latest stable version of PyTest from the Python Package Index (PyPI). After installation, the version can be verified using the command:

```
pytest --version
```

The installation process is simple and requires an active internet connection.

6.Installation Process

The installation process was carried out in the following steps:

Step 1: Install Python from the official website.

Step 2: Open Command Prompt or PowerShell.

Step 3: Check Python installation:

`python --version`

```
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar> python --version
Python 3.13.0
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar> |
```

Step 4: Install PyTest:

`pip install pytest`

```
Windows PowerShell
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar> pip install pytest
Collecting pytest
  Using cached pytest-9.0.2-py3-none-any.whl.metadata (7.6 kB)
Requirement already satisfied: colorama>=0.4 in C:\Users\Rupali More\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in C:\Users\Rupali More\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=22 in C:\Users\Rupali More\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (24.1)
Requirement already satisfied: pluggy<2, >=1.5 in C:\Users\Rupali More\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in C:\Users\Rupali More\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (2.18.0)
Using cached pytest-9.0.2-py3-none-any.whl (374 kB)
Installing collected packages: pytest
Successfully installed pytest-9.0.2
```

Step 5: Confirm successful installation:

`pytest --version`

```
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar> pytest --version
pytest 9.0.2
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar> |
```

The successful installation message confirmed that PyTest was installed properly.

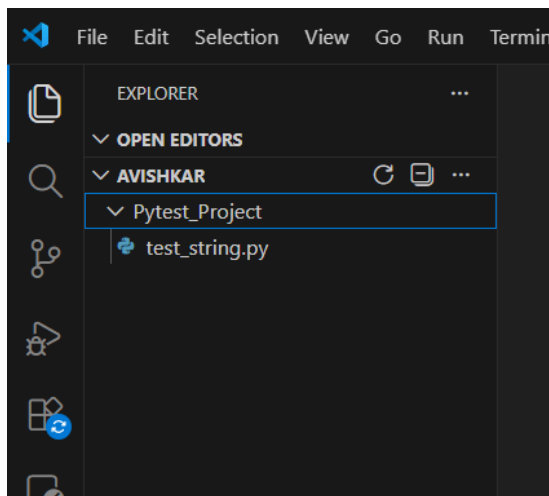
7.Configuration / Setup

A new project folder named PyTest_Project was created on the system.

Inside this folder, a test file named:

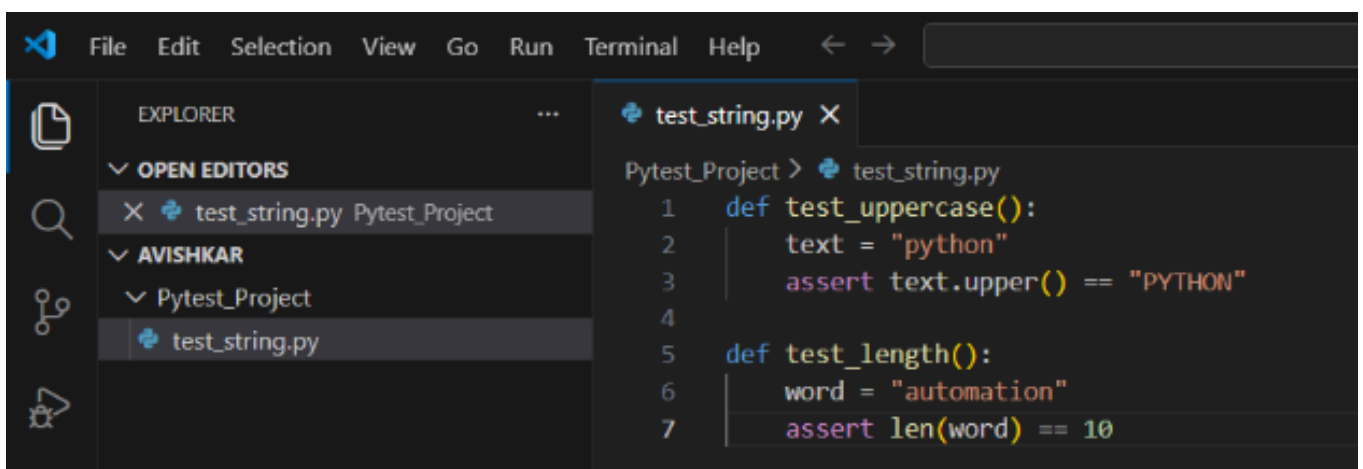
test_string.py

was created.



It is important that the file name starts with "test_" because PyTest automatically identifies such files.

The following test cases were written inside the file:



Each function starts with the keyword "test_" so that PyTest can detect it.

8.Working of the Tool

PyTest works by automatically detecting test files and executing test functions that follow a specific naming convention. Any file that begins with “test_” and any function that starts with “test_” is recognized as a test case by PyTest.

When the command pytest is executed in the terminal, the framework scans the project directory, collects all valid test cases, and runs them one by one.

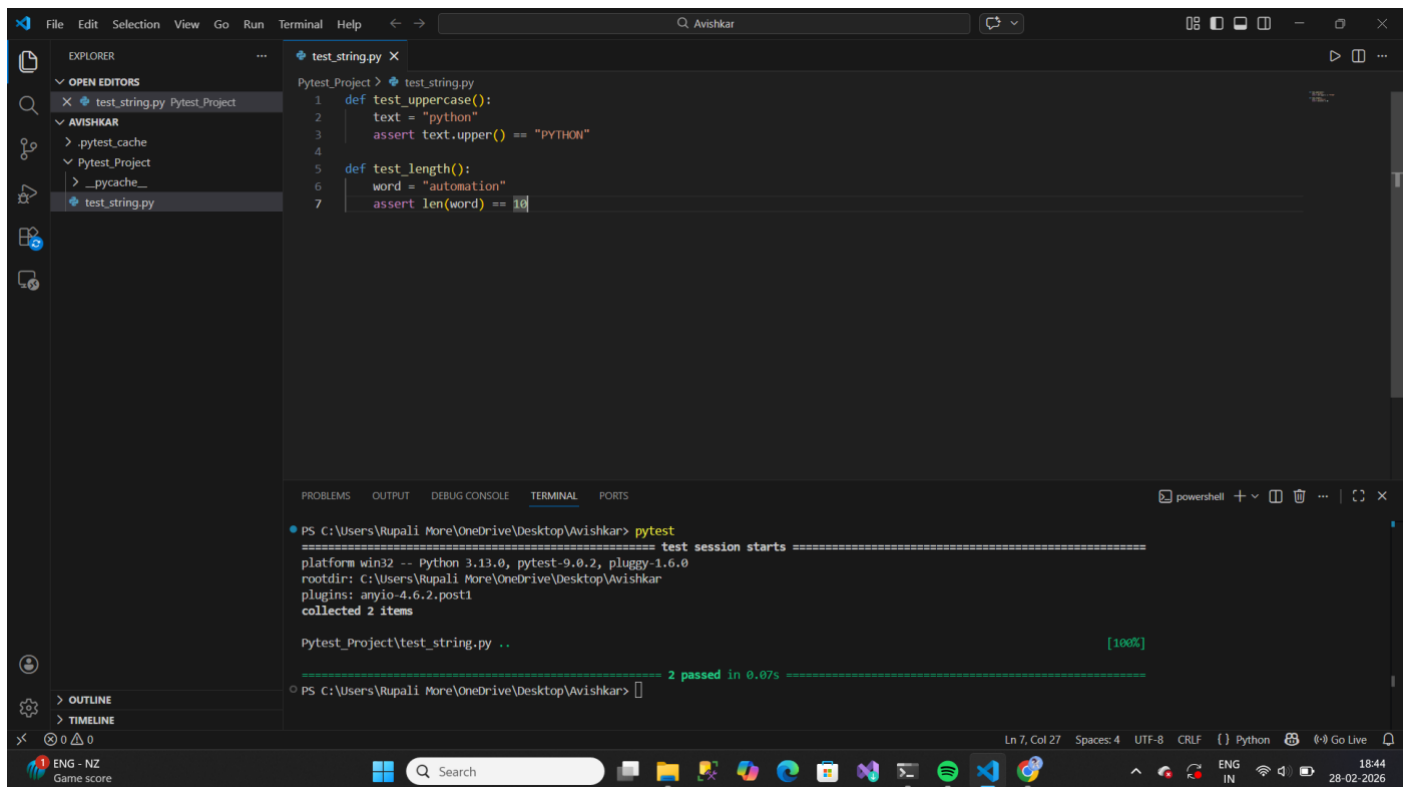
If the condition inside the assert statement is true, the test case is marked as PASSED. If the condition is false, PyTest marks the test case as FAILED and displays a detailed error message showing the expected and actual output.

The framework also displays:

- Number of test cases collected
- Number of tests passed or failed
- Execution time
- Detailed assertion errors (if any)

This makes debugging simple and efficient.

- **Successful Execution (2 Passed)**



The screenshot shows the Visual Studio Code interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'Pytest_Project' with a file 'test_string.py'. The code editor shows the following Python code:

```
1 def test_uppercase():
2     text = "python"
3     assert text.upper() == "PYTHON"
4
5 def test_length():
6     word = "automation"
7     assert len(word) == 10
```

The terminal shows the output of the pytest command:

```
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar> pytest
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Rupali More\OneDrive\Desktop\Avishkar
plugins: anyio-4.6.2.post1
collected 2 items

Pytest_Project\test_string.py ..                                     [100%]

===== 2 passed in 0.07s =====
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar>
```

Successful Execution of Test Cases (2 Passed).

- **Failed Test Case**

1 failed, 1 passed

AssertionError

```
File Edit Selection View Go Run Terminal Help
test_string.py X
Pytest_Project > test_string.py
1 def test_uppercase():
2     text = "python"
3     assert text.upper() == "PYTHON"
4
5 def test_length():
6     word = "automation"
7     assert len(word) == 8 # for failing test

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar> pytest
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.0.2, pluggy-1.6.0
rootdir: c:\Users\Rupali More\OneDrive\Desktop\Avishkar
plugins: anyio-4.6.2.post1
collected 2 items

Pytest_Project\test_string.py .F [100%]

===== FAILURES =====
test_length

def test_length():
    word = "automation"
    > assert len(word) == 8 # for failing test
    ~~~~~
    E       AssertionError: assert 10 == 8
    E       + where 10 = len('automation')

Pytest_Project\test_string.py:7: AssertionError

===== short test summary info =====
FAILED Pytest_Project\test_string.py::test_length - AssertionError: assert 10 == 8
===== 1 failed, 1 passed in 0.22s =====
PS C:\Users\Rupali More\OneDrive\Desktop\Avishkar>
```

Failed Test Case Showing Assertion Error

9. Advantages

PyTest provides several advantages:

- Easy to learn and use.
- Requires minimal code.
- Provides detailed failure output.
- Saves development time.
- Supports automation testing.
- Suitable for both small and large projects.

10. Limitations

Despite its advantages, PyTest has some limitations:

- Mainly used for Python-based applications.
- Requires proper naming conventions.
- Advanced features may be complex for beginners.
- Large projects may require additional plugins.

11.Real-World Applications

PyTest is widely used in:

- ✓ Web application testing
- ✓ API testing
- ✓ Backend system validation
- ✓ Automation testing projects
- ✓ Continuous integration systems
- ✓ Software development companies

Many organizations use PyTest to ensure code quality and reliability.

12.Challenges Faced During Installation

During the practical, the following challenges were faced:

- Initial installation errors.
- Understanding naming conventions.
- Resolving path-related issues.
- Handling assertion errors.
- Configuring project folder properly.

These challenges were resolved by verifying commands and following correct steps.

13. Conclusion

In this assignment, I learned how to install and use the PyTest testing tool. I understood how to write simple test cases and run them using the command prompt. I also learned how PyTest checks whether the program is working correctly or not.

By doing this project, I got practical knowledge about automated testing. It helped me understand the importance of testing in software development. PyTest makes testing easy, fast, and accurate.

Overall, this project improved my basic understanding of software testing and gave me confidence in using testing tools in real applications.

14. References

PyTest Official Documentation. Available at: <https://docs.pytest.org>

Python Official Website. Available at: <https://www.python.org>

Python Package Index (PyPI). Available at: <https://pypi.org>